Helm

theCylon

Introduction to Helm

- tool for managing Kubernetes packages called charts
- using Helm to manage packages on our Kubernetes cluster
- Helm can do the following:
  - Create new charts from scratch
  - Package charts into chart archive (tgz) files
  - Interact with chart repositories where charts are stored
  - Install and uninstall charts into an existing Kubernetes cluster
  - Manage the release cycle of charts that have been installed with Helm

theCylon

Introduction to Helm

- standalone Helm library encapsulates Helm logic
  - can be leveraged by different clients
- Helm
  - installs charts into Kubernetes
  - creates a new release for each installation
- need to find new charts?
  - search Helm chart repositories

theCylon

Helm Concepts

- Charts
  - a Helm package
  - bundle of information used to create instance of an application
  - contains all of the resource definitions necessary to run an application, tool, or service inside of a Kubernetes cluster
  - Kubernetes equivalent of a Homebrew formula, an Apt dpkg, or Yum
  - everytime we install a chart, a new release is created
  - a single chart can be installed multiple times into the same cluster
  - each can be independently managed and upgraded
    - helm ls

theCylon

Helm Concepts

- Repository
  - place where charts can be collected and shared
- Config
  - contains configuration information
  - merged into a packaged chart to create a releasable object
- Release
  - an instance of a chart running in a Kubernetes cluster
  - a running instance of a chart, combined with a specific config
  - One chart can often be installed many times into the same cluster
  - for each chart installation, a new release is created

theCylon

Helm Components

- an executable – implemented into two distinct parts
  - The Helm Client
  - The Helm Library
- both Helm client and library is written in the Go programming language
- library uses the Kubernetes client library to communicate with Kubernetes
- library uses REST+JSON
- stores information in Secrets located inside of Kubernetes
- does not need its own database
- Configuration files are, when possible are written in YAML

theCylon

Helm Client

- is a command-line client for end users
- client is responsible for the following
  - Local chart development
  - Managing repositories
  - Managing releases
  - Interfacing with the Helm library
    - Sending charts to be installed
    - Requesting upgrading or uninstalling of existing releases

theCylon

Helm Library

- provides the logic for executing all Helm operations
- interfaces with the Kubernetes API server
- provides the below capability:
  - Combining a chart and configuration to build a release
  - Installing charts into Kubernetes, providing subsequent release object
  - Upgrading and uninstalling charts by interacting with Kubernetes

theCylon

Helm Installation

- curl -fsSL -o get_helm.sh

  https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3
- chmod 700 get_helm.sh
- ./get_helm.sh
- helm version
- helm get -h

theCylon

Helm Search Commands

- Helm hub
  - comprises of helm charts from multiple repositories
  - search made over internet
- Helm repo
  - repositories which we have added to our local helm client
  - search is done over local data
  - no public network connection is needed

theCylon

Helm Search Commands

- 2 search patterns
  - helm search hub
  - helm search repo
- Search examples
  - helm search hub wordpress
  - helm repo add brigade https://brigadecore.github.io/charts
  - helm search repo brigade
- Helm search uses a fuzzy string matching algorithm

theCylon

Helm Search Commands

- we can type parts of words or phrases
  - helm search repo kash
- found a package we want to install?
- use helm install to install it
  - helm install happy-panda stable/mariadb
- keep track of a release's state or re-read configuration information
  - helm status happy-panda

theCylon

Helm Search Commands

- Search repo:
  - helm repo list
- Add repo
  - helm repo add stable

    https://kubernetes-charts.storage.googleapis.com/
- list the charts we can install
  - helm search repo stable
  - helm search repo stable/mysql -l
- Ensure we get the latest list of charts
  - helm repo update

theCylon

Helm Install Commands

- list features of this MySQL chart
  - helm show chart stable/mysql
- get all information about the chart
  - helm show all stable/mysql
- Install mysql using Helm's chart
  - helm install stable/mysql --generate-name
  - helm install smiling-penguin stable/mysql
  - helm install one stable/mysql --version 1.6.0
- List installed releases
  - helm ls

theCylon

Helm Install Commands

- removes all resources associated with release along with release history
  - helm uninstall smiling-penguin
  - helm uninstall mysql-1591586775 mysql-1591587000 --dry-run
  - helm uninstall mysql-1591586775 mysql-1591587000
  - helm uninstall smiling-penguin --keep-history
- Helm tracks our releases even after we've uninstalled them
- request information about that release
  - helm status smiling-penguin

theCylon

Helm Upgrade Commands

- Install mysql chart
  - helm install my-mysqlrelease stable/mysql
  - helm ls
  - helm status my-mysqlrelease
- Upgrade the release with a newer chart version when available
  - helm upgrade my-mysqlrelease stable/mysql
- upgrade to a specific version of the application
  - helm upgrade my-mysqlrelease stable/mysql --version 1.6.1
  - helm upgrade my-mysqlrelease stable/mysql --version 1.6.2
  - helm upgrade my-mysqlrelease stable/mysql --version 1.6.3

theCylon

Helm Upgrade Commands

- List the releases
  - helm ls
- history of revisions using the following command
  - helm history my-mysqlrelease
- update a parameter on an existing release
  - helm upgrade my-mysqlrelease stable/mysql --version 1.6.0 --set mysqlRootPassword="DontUseThisPassword"
- Verify the password
  - kubectl get secret --namespace default my-mysqlrelease -o jsonpath="{.data.mysql-root-password}" | base64 --decode; echo

theCylon

Helm Rollback & Delete Commands

- See history
  - helm history my-mysqlrelease
- Rollback
  - helm rollback my-mysqlrelease
  - helm rollback my-mysqlrelease 4
- Delete the release now
  - helm delete my-mysqlrelease
- Verify the release has been deleted now
  - helm ls

theCylon

Customizing Helm's Chart

- see the configurable options on a chart
  - helm show values stable/mariadb
- override any of these settings in a YAML formatted file
- then pass that file during installation
  - echo '{mariadbUser: admin, mariadbDatabase: admindb}' > config.yaml
  - helm install –f config.yaml stable/mariadb ––generate–name
- creates a default MariaDB user with the name admin
- grants this user access to a newly created admindb database
- will accept all the rest of the defaults for that chart

theCylon

Customizing Helm's Chart

- two ways to pass configuration data during install
  - --values or -f
    - Specify a YAML file with overrides
  - --set
    - Specify overrides on the command line
    - --set servers[0].port=80,servers[0].host=example
- both are used?
  - --set values are merged into --values with higher precedence

theCylon

Custom Helm's Chart

- Generate chart
  - helm create nginx
  - tree nginx

```
nginx
|-- Chart.yaml
|-- charts
|-- templates
|  |-- NOTES.txt
|  |-- _helpers.tpl
|  |-- deployment.yaml
|  |-- ingress.yaml
|  `-- service.yaml
`-- values.yaml
```

theCylon

Custom Helm's Chart

- Templates Directory
  - most important piece of the helm's chart puzzle
  - definitions for services, deployments and other Kubernetes objects
  - Helm runs each file in this directory via Go template rendering engine
    - service.yaml
    - deployment.yaml
- dry-run of a helm install and enable debug to inspect the generated definitions
  - helm install nginx-release nginx --dry-run --debug

theCylon

Custom Helm's Chart

- Notes.txt
  - gets prinited after a helm's chart is installed
- Values
  - template in service.yaml makes use of the Helm-specific objects
    - .charts
    - .values

theCylon

Custom Helm's Chart

- .charts
  - provides metadata about the chart to our definitions
  - eg: name, version etc.
- .values
  - key element of Helm charts
  - exposes configuration that can be set at the time of deployment
  - defaults for this object are defined in the values.yaml file

theCylon

Custom Helm's Chart

- changing the default value for service.port
  - helm install nginx-r --dry-run --debug nginx --set service.port=9097
- changing the value for service.type to NodePort
  - helm install nginx-r nginx --set service.type=NodePort

theCylon

Custom Helm's Chart

- copy the values.yml to myvalues.yml
  - set your own parameters
    - image
    - serviceType
  - configure all other necessary details to make it function
- Upgrade using the below command
  - helm upgrade nginx-r nginx –f myvalues.yml

theCylon

Helm Plugin

- tool that can be accessed through the helm CLI
- not part of the built-in Helm codebase
- add-on tools that integrate seamlessly with Helm
- provide a way to extend the core feature set of Helm
- can be added and removed from a Helm installation
- no impact to the core Helm tool
- can be written in any programming language
- integrate with Helm, and will show up in helm help and other places
- Installing a Plugin
  - helm plugin install <path|url>

theCylon

Helm Plugin Installation

- helm plugin install https://github.com/technosophos/helm-github
- helm plugins have a top-level directory and then a plugin.yaml file

```
$XDG_DATA_HOME/helm/plugins/
  |- keybase/
     |
     |- plugin.yaml
     |- keybase.sh
```

- core of a plugin is a simple YAML file named plugin.yaml
- helm env
- helm plugin list

theCylon

Helm Custom Plugin

- cd /home/vagrant/.cache/helm/plugins
- mkdir hello
- cd hello
- cat <<EOF> plugin.yaml

```
name: "hello"
version: "0.1.0"
usage: "Say hello"
description: |-
  This is a demonstration plugin that prints Hello and then exists.
command: "env"
EOF
```

theCylon

Helm Custom Plugin

- cd ..
- helm plugin install hello
- helm hello

theCylon

Helm Custom Plugin

- One more example
  - cd ~
  - mkdir edge
  - cd edge
  - cat <<EOF> plugin.yaml

```
name: "edge"
version: "0.1.0"
usage: "Do Something"
description: |-
  This is a demonstration plugin that executes a script and then exists.
command: "$HELM_PLUGIN_DIR/dosomething.sh"
EOF
```

theCylon

Helm Custom Plugin

- One more example
- cat <<EOF> dosomething.sh

```
#!/bin/bash
echo "Hello from a Helm plugin"
echo "PARAMS"
echo $*
echo "ENVIRONMENT"
env
echo $HELM_HOME
EOF
```

- chmod 777 dosomething.sh
- cd ..

theCylon

Helm Custom Plugin

- helm plugin install edge
- helm edge
- helm plugin remove edge

theCylon

Helm Hooks

- use hooks to
  - Execute a Job to backup a database before installing a new chart
  - execute a second job after the upgrade in order to restore data
  - Run a Job before deleting a release

theCylon

Helm Hook Type

- pre-install
- post-install
- pre-delete
- post-delete
- pre-upgrade
- post-upgrade
- pre-rollback
- post-rollback

theCylon

Helm Hook Example

```
apiVersion: batch/v1
kind: Job
metadata:
 name: "{{ .Release.Name }}"
 labels:
  app.kubernetes.io/managed-by: {{ .Release.Service | quote }}
  app.kubernetes.io/instance: {{ .Release.Name | quote }}
  app.kubernetes.io/version: {{ .Chart.AppVersion }}
  helm.sh/chart: "{{ .Chart.Name }}-{{ .Chart.Version }}"
 annotations:
  # This is what defines this resource as a hook. Without this line, the
  # job is considered part of the release.
  "helm.sh/hook": post-install
  "helm.sh/hook-weight": "-5"
  "helm.sh/hook-delete-policy": hook-succeeded
spec:
 template:
  metadata:
   name: "{{ .Release.Name }}"
   labels:
    app.kubernetes.io/managed-by: {{ .Release.Service | quote }}
    app.kubernetes.io/instance: {{ .Release.Name | quote }}
    helm.sh/chart: "{{ .Chart.Name }}-{{ .Chart.Version }}"
  spec:
   restartPolicy: Never
   containers:
   - name: post-install-job
    image: "alpine:3.3"
    command: ["/bin/sleep","{{ default "10" .Values.sleepyTime }}"]
```

theCylon

Helm Reference

- https://helm.sh/docs/topics/charts_hooks/

theCylon